Table of Contents

2.1. The class File Format
2.2. Data Types
2.3. Primitive Types and Values
2.3.1. Integral Types and Values
2.3.2. Floating-Point Types and Values
2.3.3. The returnAddress Type and Values
2.3.4. The boolean Type
2.4. Reference Types and Values
2.5. Run-Time Data Areas
2.5.1. The pc Register
2.5.2. Java Virtual Machine Stacks
2.5.3. Heap
2.5.4. Method Area
2.5.5. Run-Time Constant Pool
2.5.6. Native Method Stacks
2.6. Frames
2.6.1. Local Variables
2.6.2. Operand Stacks
2.6.3. Dynamic Linking
2.6.4. Normal Method Invocation Completion
2.6.5. Abrupt Method Invocation Completion
2.7. Representation of Objects
2.8. Floating-Point Arithmetic
2.9. Special Methods
2.9.1. Instance Initialization Methods
2.9.2. Class Initialization Methods
2.9.3. Signature Polymorphic Methods
2.10. Exceptions
2.11. Instruction Set Summary
2.11.1. Types and the Java Virtual Machine
2.11.2. Load and Store Instructions
2.11.3. Arithmetic Instructions
2.11.4. Type Conversion Instructions
2.11.5. Object Creation and Manipulation
2.11.6. Operand Stack Management Instructions
2.11.7. Control Transfer Instructions
2.11.8. Method Invocation and Return Instructions
2.11.9. Throwing Exceptions
2.11.10. Synchronization
2.12. Class Libraries
2.13. Public Design, Private Implementation

Chapter 2. The Structure of the Java Virtual Machine

This document specifies an abstract machine. It does not describe any particular implementation of the Java Virtual Machine.

To implement the Java Virtual Machine correctly, you need only be able to read the `class` file format and correctly perform the operations specified therein. Implementation details that are not part of the Java Virtual Machine's specification would unnecessarily constrain the creativity of implementors. For example, the memory layout of run-time data areas, the garbage-collection algorithm used, and any internal optimization of the Java Virtual Machine instructions (for example, translating them into machine code) are left to the discretion of the implementor.

All references to Unicode in this specification are given with respect to *The Unicode Standard, Version 17.0.0*, available at `https://www.unicode.org/`.

2.1. The `class` File Format

Compiled code to be executed by the Java Virtual Machine is represented using a hardware- and operating system-independent binary format, typically (but not necessarily) stored in a file, known as the `class` file format. The `class` file format precisely defines the representation of a class or interface, including details such as byte ordering that might be taken for granted in a platform-specific object file format.

Chapter 4, "The `class` File Format", covers the `class` file format in detail.

2.2. Data Types

Like the Java programming language, the Java Virtual Machine operates on two kinds of types: *primitive types* and *reference types*. There are, correspondingly, two kinds of values that can be stored in variables, passed as arguments, returned by methods, and operated upon: *primitive values* and *reference values*.

The Java Virtual Machine expects that nearly all type checking is done prior to run time, typically by a compiler, and does not have to be done by the Java Virtual Machine itself. Values of primitive types need not be tagged or otherwise be inspectable to determine their types at run time, or to be distinguished from values of reference types. Instead, the instruction set of the Java Virtual Machine distinguishes its operand types using instructions intended to operate on values of specific types. For instance, *iadd*, *ladd*, *fadd*, and *dadd* are all Java Virtual Machine instructions that add two numeric values and produce numeric results, but each is specialized for its operand type: `int`, `long`, `float`, and `double`, respectively. For a summary of type support in the Java Virtual Machine instruction set, see §2.11.1.

The Java Virtual Machine contains explicit support for objects. An object is either a dynamically allocated class instance or an array. A reference to an object is considered to have Java Virtual Machine type `reference`. References are polymorphic: a single reference may also be a value of multiple class types, interface types, or array types. Values of type `reference` can be thought of as pointers to objects. More than one reference to an object may exist. Objects are always operated on, passed, and tested via values of type `reference`.

2.3. Primitive Types and Values

The primitive data types supported by the Java Virtual Machine are the *numeric types*, the `boolean` type (§2.3.4), and the `returnAddress` type (§2.3.3).

The numeric types consist of the *integral types* (§2.3.1) and the *floating-point types* (§2.3.2).

The integral types are:

- `byte`, whose values are 8-bit signed two's-complement integers, and whose default value is zero
- `short`, whose values are 16-bit signed two's-complement integers, and whose default value is zero
- `int`, whose values are 32-bit signed two's-complement integers, and whose default value is zero
- `long`, whose values are 64-bit signed two's-complement integers, and whose default value is zero
- `char`, whose values are 16-bit unsigned integers representing Unicode code points in the Basic Multilingual Plane, encoded with UTF-16, and whose default value is the null code point (`'\u0000'`)

The floating-point types are:

- `float`, whose values exactly correspond to the values representable in the 32-bit IEEE 754 binary32 format, and whose default value is positive zero
- `double`, whose values exactly correspond to the values of the 64-bit IEEE 754 binary64 format, and whose default value is positive zero

The values of the `boolean` type encode the truth values `true` and `false`, and the default value is `false`.

The First Edition of The Java® Virtual Machine Specification did not consider `boolean` to be a Java Virtual Machine type. However, `boolean` values do have limited support in the Java Virtual Machine. The Second Edition of The Java® Virtual Machine Specification clarified the issue by treating `boolean` as a type.

The values of the `returnAddress` type are pointers to the opcodes of Java Virtual Machine instructions. Of the primitive types, only the `returnAddress` type is not directly associated with a Java programming language type.

2.3.1. Integral Types and Values

The values of the integral types of the Java Virtual Machine are:

- For `byte`, from -128 to 127 (-2^7 to $2^7 - 1$), inclusive
- For `short`, from -32768 to 32767 (-2^{15} to $2^{15} - 1$), inclusive
- For `int`, from -2147483648 to 2147483647 (-2^{31} to $2^{31} - 1$), inclusive
- For `long`, from -9223372036854775808 to 9223372036854775807 (-2^{63} to $2^{63} - 1$), inclusive
- For `char`, from 0 to 65535 inclusive

2.3.2. Floating-Point Types and Values

The floating-point types are `float` and `double`, which are conceptually associated with the 32-bit binary32 and 64-bit binary64 floating-point formats for IEEE 754 values and operations, as specified in the IEEE 754 Standard (JLS §1.7).

In Java SE 15 and later, the Java Virtual Machine uses the 2019 version of the IEEE 754 Standard. Prior to Java SE 15, the Java Virtual Machine used the 1985 version of the IEEE 754 Standard, where the binary32 format was known as the single format and the binary64 format was known as the double format.

IEEE 754 includes not only positive and negative numbers that consist of a sign and magnitude, but also positive and negative zeros, positive and negative *infinities*, and special *Not-a-Number* values (hereafter abbreviated NaN). A NaN value is used to represent the result of certain invalid operations such as dividing zero by zero. NaN constants of both `float` and `double` type are predefined as `Float.NaN` and `Double.NaN`.

The finite nonzero values of a floating-point type can all be expressed in the form $s \cdot m \cdot 2^{(e - N + 1)}$, where:

- s is +1 or -1,
- m is a positive integer less than 2^N,
- e is an integer between $E_{min} = -(2^{K-1}-2)$ and $E_{max} = 2^{K-1}-1$, inclusive, and
- `N` and K are parameters that depend on the type.

Some values can be represented in this form in more than one way. For example, supposing that a value `v` of a floating-point type might be represented in this form using certain values for s, m, and e, then if it happened that m were even and e were less than 2^{K-1}, one could halve m and increase e by 1 to produce a second representation for the same value `v`.

A representation in this form is called *normalized* if $m \geq 2^{N-1}$; otherwise the representation is said to be *subnormal*. If a value of a floating-point type cannot be represented in such a way that $m \geq 2^{N-1}$, then the value is said to be a *subnormal value*, because its magnitude is below the magnitude of the smallest normalized value.

The constraints on the parameters `N` and K (and on the derived parameters E_{min} and E_{max}) for `float` and `double` are summarized in Table 2.3.2-A.

Table 2.3.2-A. Floating-point parameters

Parameter	`float`	`double`
`N`	24	53
K	8	11
E_{max}	+127	+1023
E_{min}	-126	-1022

Except for NaN, floating-point values are *ordered*. When arranged from smallest to largest, they are negative infinity, negative finite nonzero values, positive and negative zero, positive finite nonzero values, and positive infinity.

IEEE 754 allows multiple distinct NaN values for each of its binary32 and binary64 floating-point formats. However, the Java SE Platform generally treats NaN values of a given floating-point type as though collapsed into a single canonical value, and hence this specification normally refers to an arbitrary NaN as though to a canonical value.

Under IEEE 754, a floating-point operation with non-NaN arguments may generate a NaN result. IEEE 754 specifies a set of NaN bit patterns, but does not mandate which particular NaN bit pattern is used to represent a NaN result; this is left to the hardware architecture. A programmer can create NaNs with different bit patterns to encode, for example, retrospective diagnostic information. These NaN values can be created with the `Float.intBitsToFloat` and `Double.longBitsToDouble` methods for `float` and `double`, respectively. Conversely, to inspect the bit patterns of NaN values, the `Float.floatToRawIntBits` and `Double.doubleToRawLongBits` methods can be used for `float` and `double`, respectively.

Positive zero and negative zero compare equal, but there are other operations that can distinguish them; for example, dividing `1.0` by `0.0` produces positive infinity, but dividing `1.0` by `-0.0` produces negative infinity.

NaN is *unordered*, so numerical comparisons and tests for numerical equality have the value `false` if either or both of their operands are NaN. In particular, a test for numerical equality of a value against itself has the value `false` if and only if the value is NaN. A test for numerical inequality has the value `true` if either operand is NaN.

2.3.3. The `returnAddress` Type and Values

The `returnAddress` type is used by the Java Virtual Machine's *jsr*, *ret*, and *jsr_w* instructions (§*jsr*, §*ret*, §*jsr_w*). The values of the `returnAddress` type are pointers to the opcodes of Java Virtual Machine instructions. Unlike the numeric primitive types, the `returnAddress` type does not correspond to any Java programming language type and cannot be modified by the running program.

2.3.4. The `boolean` Type

Although the Java Virtual Machine defines a `boolean` type, it only provides very limited support for it. There are no Java Virtual Machine instructions solely dedicated to operations on `boolean` values. Instead, expressions in the Java programming language that operate on `boolean` values are compiled to use values of the Java Virtual Machine `int` data type.

The Java Virtual Machine does directly support `boolean` arrays. Its *newarray* instruction (§*newarray*) enables creation of `boolean` arrays. Arrays of type `boolean` are accessed and modified using the `byte` array instructions *baload* and *bastore* (§*baload*, §*bastore*).

In Oracle's Java Virtual Machine implementation, `boolean` arrays in the Java programming language are encoded as Java Virtual Machine `byte` arrays, using 8 bits per `boolean` element.

The Java Virtual Machine encodes `boolean` array components using `1` to represent `true` and `0` to represent `false`. Where Java programming language `boolean` values are mapped by compilers to values of Java Virtual Machine type `int`, the compilers must use the same encoding.

2.4. Reference Types and Values

There are three kinds of `reference` types: class types, array types, and interface types. Their values are references to dynamically created class instances, arrays, or class instances or arrays that implement interfaces, respectively.

An array type consists of a *component type* with a single dimension (whose length is not given by the type). The component type of an array type may itself be an array type. If, starting from any array type, one considers its component type, and then (if that is also an array type) the component type of that type, and so on, eventually one must reach a component type that is not an array type; this is called the *element type* of the array type. The element type of an array type is necessarily either a primitive type, or a class type, or an interface type.

A `reference` value may also be the special null reference, a reference to no object, which will be denoted here by `null`. The `null` reference initially has no run-time type, but may be cast to any type. The default value of a `reference` type is `null`.

This specification does not mandate a concrete value encoding `null`.

2.5. Run-Time Data Areas

The Java Virtual Machine defines various run-time data areas that are used during execution of a program. Some of these data areas are created on Java Virtual Machine start-up and are destroyed only when the Java Virtual Machine terminates. Other data areas are per thread. Per-thread data areas are created when a thread is created and destroyed when the thread terminates.

2.5.1. The `pc` Register

The Java Virtual Machine can support many threads of execution at once (JLS §17). Each Java Virtual Machine thread has its own `pc` (program counter) register. At any point, each Java Virtual Machine thread is executing the code of a single method, namely the current method (§2.6) for that thread. If that method is not `native`, the `pc` register contains the address of the Java Virtual Machine instruction currently being executed. If the method currently being executed by the thread is `native`, the value of the Java Virtual Machine's `pc` register is undefined. The Java Virtual Machine's `pc` register is wide enough to hold a `returnAddress` or a native pointer on the specific platform.

2.5.2. Java Virtual Machine Stacks

Each Java Virtual Machine thread has a private *Java Virtual Machine stack*, created at the same time as the thread. A Java Virtual Machine stack stores frames (§2.6). A Java Virtual Machine stack is analogous to the stack of a conventional language such as C: it holds local variables and partial results, and plays a part in method invocation and return. Because the Java Virtual Machine stack is never manipulated directly except to push and pop frames, frames may be heap allocated. The memory for a Java Virtual Machine stack does not need to be contiguous.

In the First Edition of The Java® Virtual Machine Specification, the Java Virtual Machine stack was known as the Java stack.

This specification permits Java Virtual Machine stacks either to be of a fixed size or to dynamically expand and contract as required by the computation. If the Java Virtual Machine stacks are of a fixed size, the size of each Java Virtual Machine stack may be chosen independently when that stack is created.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of Java Virtual Machine stacks, as well as, in the case of dynamically expanding or contracting Java Virtual Machine stacks, control over the maximum and minimum sizes.

The following exceptional conditions are associated with Java Virtual Machine stacks:

- **If the computation in a thread requires a larger Java Virtual Machine stack than is permitted, the Java Virtual Machine throws a `StackOverflowError`.**
- **If Java Virtual Machine stacks can be dynamically expanded, and expansion is attempted but insufficient memory can be made available to effect the expansion, or if insufficient memory can be made available to create the initial Java Virtual Machine stack for a new thread, the Java Virtual Machine throws an `OutOfMemoryError`.**

2.5.3. Heap

The Java Virtual Machine has a *heap* that is shared among all Java Virtual Machine threads. The heap is the run-time data area from which memory for all class instances and arrays is allocated.

The heap is created on virtual machine start-up. Heap storage for objects is reclaimed by an automatic storage management system (known as a *garbage collector*); objects are never explicitly deallocated. The Java Virtual Machine assumes no particular type of automatic storage management system, and the storage management technique may be chosen according to the implementor's system requirements. The heap may be of a fixed size or may be expanded as required by the computation and may be contracted if a larger heap becomes unnecessary. The memory for the heap does not need to be contiguous.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of the heap, as well as, if the heap can be dynamically expanded or contracted, control over the maximum and minimum heap size.

The following exceptional condition is associated with the heap:

- **If a computation requires more heap than can be made available by the automatic storage management system, the Java Virtual Machine throws an `OutOfMemoryError`.**

2.5.4. Method Area

The Java Virtual Machine has a *method area* that is shared among all Java Virtual Machine threads. The method area is analogous to the storage area for compiled code of a conventional language or analogous to the "text" segment in an operating system process. It stores per-class structures such as the run-time constant pool, field and method data, and the code for methods and constructors, including the special methods used in class and interface initialization and in instance initialization (§2.9).

The method area is created on virtual machine start-up. Although the method area is logically part of the heap, simple implementations may choose not to either garbage collect or compact it. This specification does not mandate the location of the method area or the policies used to manage compiled code. The method area may be of a fixed size or may be expanded as required by the computation and may be contracted if a larger method area becomes unnecessary. The memory for the method area does not need to be contiguous.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of the method area, as well as, in the case of a varying-size method area, control over the maximum and minimum method area size.

The following exceptional condition is associated with the method area:

- **If memory in the method area cannot be made available to satisfy an allocation request, the Java Virtual Machine throws an `OutOfMemoryError`.**

2.5.5. Run-Time Constant Pool

A *run-time constant pool* is a per-class or per-interface run-time representation of the `constant_pool` table in a `class` file (§4.4). It contains several kinds of constants, ranging from numeric literals known at compile-time to method and field references that must be resolved at run-time. The run-time constant pool serves a function similar to that of a symbol table for a conventional programming language, although it contains a wider range of data than a typical symbol table.

Each run-time constant pool is allocated from the Java Virtual Machine's method area (§2.5.4). The run-time constant pool for a class or interface is constructed when the class or interface is created (§5.3) by the Java Virtual Machine.

The following exceptional condition is associated with the construction of the run-time constant pool for a class or interface:

- **When creating a class or interface, if the construction of the run-time constant pool requires more memory than can be made available in the method area of the Java Virtual Machine, the Java Virtual Machine throws an `OutOfMemoryError`.**

See §5 (Loading, Linking, and Initializing) for information about the construction of the run-time constant pool.

2.5.6. Native Method Stacks

An implementation of the Java Virtual Machine may use conventional stacks, colloquially called "C stacks," to support `native` methods (methods written in a language other than the Java programming language). Native method stacks may also be used by the implementation of an interpreter for the Java Virtual Machine's instruction set in a language such as C. Java Virtual Machine implementations that cannot load `native` methods and that do not themselves rely on conventional stacks need not supply native method stacks. If supplied, native method stacks are typically allocated per thread when each thread is created.

This specification permits native method stacks either to be of a fixed size or to dynamically expand and contract as required by the computation. If the native method stacks are of a fixed size, the size of each native method stack may be chosen independently when that stack is created.

A Java Virtual Machine implementation may provide the programmer or the user control over the initial size of the native method stacks, as well as, in the case of varying-size native method stacks, control over the maximum and minimum method stack sizes.

The following exceptional conditions are associated with native method stacks:

- **If the computation in a thread requires a larger native method stack than is permitted, the Java Virtual Machine throws a `StackOverflowError`.**
- **If native method stacks can be dynamically expanded and native method stack expansion is attempted but insufficient memory can be made available, or if insufficient memory can be made available to create the initial native method stack for a new thread, the Java Virtual Machine throws an `OutOfMemoryError`.**

2.6. Frames

A *frame* is used to store data and partial results, as well as to perform dynamic linking, return values for methods, and dispatch exceptions.

A new frame is created each time a method is invoked. A frame is destroyed when its method invocation completes, whether that completion is normal or abrupt (it throws an uncaught exception). Frames are allocated from the Java Virtual Machine stack (§2.5.2) of the thread creating the frame. Each frame has its own array of local variables (§2.6.1), its own operand stack (§2.6.2), and a reference to the run-time constant pool (§2.5.5) of the class of the current method.

A frame may be extended with additional implementation-specific information, such as debugging information.

The sizes of the local variable array and the operand stack are determined at compile-time and are supplied along with the code for the method associated with the frame (§4.7.3). Thus the size of the frame data structure depends only on the implementation of the Java Virtual Machine, and the memory for these structures can be allocated simultaneously on method invocation.

Only one frame, the frame for the executing method, is active at any point in a given thread of control. This frame is referred to as the *current frame*, and its method is known as the *current method*. The class in which the current method is defined is the *current class*. Operations on local variables and the operand stack are typically with reference to the current frame.

A frame ceases to be current if its method invokes another method or if its method completes. When a method is invoked, a new frame is created and becomes current when control transfers to the new method. On method return, the current frame passes back the result of its method invocation, if any, to the previous frame. The current frame is then discarded as the previous frame becomes the current one.

Note that a frame created by a thread is local to that thread and cannot be referenced by any other thread.

2.6.1. Local Variables

Each frame (§2.6) contains an array of variables known as its *local variables*. The length of the local variable array of a frame is determined at compile-time and supplied in the binary representation of a class or interface along with the code for the method associated with the frame (§4.7.3).

A single local variable can hold a value of type `int`, `float`, `reference`, or `returnAddress`. A pair of local variables can hold a value of type `long` or `double`.

Local variables are addressed by indexing. The index of the first local variable is zero. An integer is considered to be an index into the local variable array if and only if that integer is between zero and one less than the size of the local variable array.

A value of type `long` or type `double` occupies two consecutive local variables. Such a value may only be addressed using the lesser index. For example, a value of type `double` stored in the local variable array at index *n* actually occupies the local variables with indices *n* and *n*+1; however, the local variable at index *n*+1 cannot be loaded from. It can be stored into. However, doing so invalidates the contents of local variable *n*.

The Java Virtual Machine does not require *n* to be even. In intuitive terms, values of types `long` and `double` need not be 64-bit aligned in the local variables array. Implementors are free to decide the appropriate way to represent such values using the two local variables reserved for the value.

The Java Virtual Machine uses local variables to pass parameters on method invocation. On class method invocation, any parameters are passed in consecutive local variables starting from local variable *0*. On instance method invocation, local variable *0* is always used to pass a reference to the object on which the instance method is being invoked (`this` in the Java programming language). Any parameters are subsequently passed in consecutive local variables starting from local variable *1*.

2.6.2. Operand Stacks

Each frame (§2.6) contains a last-in-first-out (LIFO) stack known as its *operand stack*. The maximum depth of the operand stack of a frame is determined at compile-time and is supplied along with the code for the method associated with the frame (§4.7.3).

Where it is clear by context, we will sometimes refer to the operand stack of the current frame as simply the operand stack.

The operand stack is empty when the frame that contains it is created. The Java Virtual Machine supplies instructions to load constants or values from local variables or fields onto the operand stack. Other Java Virtual Machine instructions take operands from the operand stack, operate on them, and push the result back onto the operand stack. The operand stack is also used to prepare parameters to be passed to methods and to receive method results.

For example, the *iadd* instruction (§*iadd*) adds two `int` values together. It requires that the `int` values to be added be the top two values of the operand stack, pushed there by previous instructions. Both of the `int` values are popped from the operand stack. They are added, and their sum is pushed back onto the operand stack. Subcomputations may be nested on the operand stack, resulting in values that can be used by the encompassing computation.

Each entry on the operand stack can hold a value of any Java Virtual Machine type, including a value of type `long` or type `double`.

Values from the operand stack must be operated upon in ways appropriate to their types. It is not possible, for example, to push two `int` values and subsequently treat them as a `long` or to push two `float` values and subsequently add them with an *iadd* instruction. A small number of Java Virtual Machine instructions (the *dup* instructions (§*dup*) and *swap* (§*swap*)) operate on run-time data areas as raw values without regard to their specific types; these instructions are defined in such a way that they cannot be used to modify or break up individual values. These restrictions on operand stack manipulation are enforced through `class` file verification (§4.10).

At any point in time, an operand stack has an associated depth, where a value of type `long` or `double` contributes two units to the depth and a value of any other type contributes one unit.

2.6.3. Dynamic Linking

Each frame (§2.6) contains a reference to the run-time constant pool (§2.5.5) for the type of the current method to support *dynamic linking* of the method code. The `class` file code for a method refers to methods to be invoked and variables to be accessed via symbolic references. Dynamic linking translates these symbolic method references into concrete method references, loading classes as necessary to resolve as-yet-undefined symbols, and translates variable accesses into appropriate offsets in storage structures associated with the run-time location of these variables.

This late binding of the methods and variables makes changes in other classes that a method uses less likely to break this code.

2.6.4. Normal Method Invocation Completion

A method invocation *completes normally* if that invocation does not cause an exception (§2.10) to be thrown, either directly from the Java Virtual Machine or as a result of executing an explicit `throw` statement. If the invocation of the current method completes normally, then a value may be returned to the invoking method. This occurs when the invoked method executes one of the return instructions (§2.11.8), the choice of which must be appropriate for the type of the value being returned (if any).

The current frame (§2.6) is used in this case to restore the state of the invoker, including its local variables and operand stack, with the program counter of the invoker appropriately incremented to skip past the method invocation instruction. Execution then continues normally in the invoking method's frame with the returned value (if any) pushed onto the operand stack of that frame.

2.6.5. Abrupt Method Invocation Completion

A method invocation *completes abruptly* if execution of a Java Virtual Machine instruction within the method causes the Java Virtual Machine to throw an exception (§2.10), and that exception is not handled within the method. Execution of an *athrow* instruction (§*athrow*) also causes an exception to be explicitly thrown and, if the exception is not caught by the current method, results in abrupt method invocation completion. A method invocation that completes abruptly never returns a value to its invoker.

2.7. Representation of Objects

The Java Virtual Machine does not mandate any particular internal structure for objects.

In some of Oracle's implementations of the Java Virtual Machine, a reference to a class instance is a pointer to a handle that is itself a pair of pointers: one to a table containing the methods of the object and a pointer to the `Class` object that represents the type of the object, and the other to the memory allocated from the

heap for the object data.

2.8. Floating-Point Arithmetic

The Java Virtual Machine incorporates a subset of the floating-point arithmetic specified in the IEEE 754 Standard (JLS §1.7).

In Java SE 15 and later, the Java Virtual Machine uses the 2019 version of the IEEE 754 Standard. Prior to Java SE 15, the Java Virtual Machine used the 1985 version of the IEEE 754 Standard, where the binary32 format was known as the single format and the binary64 format was known as the double format.

Many of the Java Virtual Machine instructions for arithmetic (§2.11.3) and type conversion (§2.11.4) work with floating-point numbers. These instructions typically correspond to IEEE 754 operations (Table 2.8-A), except for certain instructions described below.

Table 2.8-A. Correspondence with IEEE 754 operations

Instruction	IEEE 754 operation
dcmp<op> (§*dcmp<op>*), *fcmp<op>* (§*fcmp<op>*)	compareQuietLess, compareQuietLessEqual, compareQuietGreater, compareQuietGreaterEqual, compareQuietEqual, compareQuietNotEqual
dadd (§*dadd*), *fadd* (§*fadd*)	addition
dsub (§*dsub*), *fsub* (§*fsub*)	subtraction
dmul (§*dmul*), *fmul* (§*fmul*)	multiplication
ddiv (§*ddiv*), *fdiv* (§*fdiv*)	division
dneg (§*dneg*), *fneg* (§*fneg*)	negate
i2d (§*i2d*), *i2f* (§*i2f*), *l2d* (§*l2d*), *l2f* (§*l2f*)	convertFromInt
d2i (§*d2i*), *d2l* (§*d2l*), *f2i* (§*f2i*), *f2l* (§*f2l*)	convertToIntegerTowardZero
d2f (§*d2f*), *f2d* (§*f2d*)	convertFormat

The key differences between the floating-point arithmetic supported by the Java Virtual Machine and the IEEE 754 Standard are:

- The floating-point remainder instructions *drem* (§*drem*) and *frem* (§*frem*) do not correspond to the IEEE 754 remainder operation. The instructions are based on an implied division using the round toward zero rounding policy; the IEEE 754 remainder is instead based on an implied division using the round to nearest rounding policy. (Rounding policies are discussed below.)
- The floating-point negate instructions *dneg* (§*dneg*) and *fneg* (§*fneg*) do not correspond precisely to the IEEE 754 negate operation. In particular, the instructions do not require the sign bit of a NaN operand to be inverted.
- The floating-point instructions of the Java Virtual Machine do not throw exceptions, trap, or otherwise signal the IEEE 754 exceptional conditions of invalid operation, division by zero, overflow, underflow, or inexact.
- The Java Virtual Machine does not support IEEE 754 signaling floating-point comparisons, and has no signaling NaN value.
- IEEE 754 includes rounding-direction attributes that do not correspond to a rounding policy in the Java Virtual Machine. The Java Virtual Machine does not provide any means to change the rounding policy used by a given floating-point instruction.
- The Java Virtual Machine does not support the binary32 extended and binary64 extended floating-point formats defined by IEEE 754. Neither extended range nor extended precision beyond those specified for the `float` and `double` types may be used when operating on or storing floating-point values.

Some IEEE 754 operations without corresponding instructions in the Java Virtual Machine are provided via methods in the `Math` and `StrictMath` classes, including the `sqrt` method for the IEEE 754 squareRoot operation, the `fma` method for the IEEE 754 fusedMultiplyAdd operation, and the `IEEEremainder` method for the IEEE 754 remainder operation.

The Java Virtual Machine requires support of IEEE 754 *subnormal* floating-point numbers and *gradual underflow*, which make it easier to prove desirable properties of particular numerical algorithms.

Floating-point arithmetic is an approximation to real arithmetic. While there are an infinite number of real numbers, a particular floating-point format only has a finite number of values. In the Java Virtual Machine, a *rounding policy* is a function used to map from a real number to a floating-point value in a given format. For real numbers in the representable range of a floating-point format, a continuous segment of the real number line is mapped to a single floating-point value. The real number whose value is numerically equal to a floating-point value is mapped to that floating-point value; for example, the real number 1.5 is mapped to the floating-point value 1.5 in a given format. The Java Virtual Machine defines two rounding policies, as follows:

- The *round to nearest* rounding policy applies to all floating-point instructions except for (i) conversion to an integer value and (ii) remainder. Under the round to nearest rounding policy, inexact results must be rounded to the representable value nearest to the infinitely precise result; if the two nearest representable values are equally near, then the value whose least significant bit is zero is chosen.

  The round to nearest rounding policy corresponds to the default rounding-direction attribute for binary arithmetic in IEEE 754, *roundTiesToEven*.

  The roundTiesToEven rounding-direction attribute was known as the "round to nearest" rounding mode in the 1985 version of the IEEE 754 Standard. The rounding policy in the Java Virtual Machine is named after this rounding mode.

- The *round toward zero* rounding policy applies to (i) conversion of a floating-point value to an integer value by the *d2i*, *d2l*, *f2i*, and *f2l* instructions (§*d2i*, §*d2l*, §*f2i*, §*f2l*), and (ii) the floating-point remainder instructions *drem* and *frem* (§*drem*, §*frem*). Under the round toward zero rounding policy, inexact results are rounded to the nearest representable value that is not greater in magnitude than the infinitely precise result. For conversion to integer, the round toward zero rounding policy is equivalent to truncation where fractional significand bits are discarded.

  The round toward zero rounding policy corresponds to the *roundTowardZero* rounding-direction attribute for binary arithmetic in IEEE 754.

  The roundTowardZero rounding-direction attribute was known as the "round toward zero" rounding mode in the 1985 version of the IEEE 754 Standard. The rounding policy in the Java Virtual Machine is named after this rounding mode.

The Java Virtual Machine requires that every floating-point instruction rounds its floating-point result to the result precision. The rounding policy used by each instruction is either round to nearest or round toward zero, as specified above.

Java 1.0 and 1.1 required strict evaluation of floating-point expressions. Strict evaluation means that each `float` operand corresponds to a value representable in the IEEE 754 binary32 format, each `double` operand corresponds to a value representable in the IEEE 754 binary64 format, and each floating-point operator with a corresponding IEEE 754 operation matches the IEEE 754 result for the same operands.

Strict evaluation provides predictable results, but caused performance problems in the Java Virtual Machine implementations for some processor families common in the Java 1.0/1.1 era. Consequently, in Java 1.2 through Java SE 16, the Java SE Platform allowed a Java Virtual Machine implementation to have one or two value sets associated with each floating-point type. The `float` type was associated with the float value set and the float-extended-exponent value set, while the `double` type was associated with the double value set and the double-extended-exponent value set. The float value set corresponded to the values representable in the IEEE 754 binary32 format; the float-extended-exponent value set had the same number of precision bits but larger exponent range. Similarly, the double value set corresponded to the values representable in the IEEE 754 binary64 format; the double-extended-exponent value set had the same number of precision bits but larger exponent range. Allowing use of the extended-exponent value sets by default ameliorated the performance problems on some processor families.

For compatibility, Java 1.2 allowed a `class` file to forbid an implementation from using the extended-exponent value sets. A `class` file expressed this by setting the `ACC_STRICT` flag on the declaration of a method. `ACC_STRICT` constrained the floating-point semantics of the method's instructions to use the float value set for `float` operands and the double value set for `double` operands, ensuring the results of such instructions were fully predictable. Methods flagged as `ACC_STRICT` thus had the same floating-point semantics as specified in Java 1.0 and 1.1.

In Java SE 17 and later, the Java SE Platform always requires strict evaluation of floating-point expressions. Newer members of the processor families that had performance problems implementing strict evaluation no longer have that difficulty. This specification no longer associates `float` and `double` with the four value sets described above, and the `ACC_STRICT` flag no longer affects the evaluation of floating-point operations. For compatibility, the bit pattern assigned to denote `ACC_STRICT` in a `class` file whose major version number is 46-60 is unassigned (that is, does not denote any flag) in a `class` file whose major version number is greater than 60 (§4.6). Future versions of the Java Virtual Machine may assign a different meaning to the bit pattern in future `class` files.

2.9. Special Methods

2.9.1. Instance Initialization Methods

A class has zero or more *instance initialization methods*, each typically corresponding to a constructor written in the Java programming language.

A method is an instance initialization method if all of the following are true:

- It is defined in a class (not an interface).
- It has the special name `<init>`.
- It is `void` (§4.3.3).

In a class, any non-`void` method named `<init>` is not an instance initialization method. In an interface, any method named `<init>` is not an instance initialization method. Such methods cannot be invoked by any Java Virtual Machine instruction (§4.4.2, §4.9.2) and are rejected by format checking (§4.6, §4.8).

The declaration and use of an instance initialization method is constrained by the Java Virtual Machine. For the declaration, the method's `access_flags` item and `code` array are constrained (§4.6, §4.9.2). For a use, an instance initialization method may be invoked only by the *invokespecial* instruction on an uninitialized class instance (§4.10.1.9).

Because the name `<init>` is not a valid identifier in the Java programming language, it cannot be used directly in a program written in the Java programming language.

2.9.2. Class Initialization Methods

A class or interface has at most one *class or interface initialization method* and is initialized by the Java Virtual Machine invoking that method (§5.5).

A method is a *class or interface initialization method* if all of the following are true:

- It has the special name `<clinit>`.
- It is `void` (§4.3.3).
- In a `class` file whose version number is 51.0 or above, the method has its `ACC_STATIC` flag set and takes no arguments (§4.6).

  The requirement for `ACC_STATIC` was introduced in Java SE 7, and for taking no arguments in Java SE 9. In a class file whose version number is 50.0 or below, a method named `<clinit>` that is `void` is considered the class or interface initialization method regardless of the setting of its `ACC_STATIC` flag or whether it takes arguments.

Other methods named `<clinit>` in a `class` file are not class or interface initialization methods. They are never invoked by the Java Virtual Machine itself, cannot be invoked by any Java Virtual Machine instruction (§4.9.1), and are rejected by format checking (§4.6, §4.8).

Because the name `<clinit>` is not a valid identifier in the Java programming language, it cannot be used directly in a program written in the Java programming language.

2.9.3. Signature Polymorphic Methods

A method is *signature polymorphic* if all of the following are true:

- It is declared in the `java.lang.invoke.MethodHandle` class or the `java.lang.invoke.VarHandle` class.
- It has a single formal parameter of type `Object[]`.
- It has the `ACC_VARARGS` and `ACC_NATIVE` flags set.

The Java Virtual Machine gives special treatment to signature polymorphic methods in the *invokevirtual* instruction (§*invokevirtual*), in order to effect invocation of a *method handle* or to effect access to a variable referenced by an instance of `java.lang.invoke.VarHandle`.

A method handle is a dynamically strongly typed and directly executable reference to an underlying method, constructor, field, or similar low-level operation (§5.4.3.5), with optional transformations of arguments or return values. An instance of `java.lang.invoke.VarHandle`

is a dynamically strongly typed reference to a variable or family of variables, including `static` fields, non-`static` fields, array elements, or components of an off-heap data structure. See the `java.lang.invoke` package in the Java SE Platform API for more information.

2.10. Exceptions

An exception in the Java Virtual Machine is represented by an instance of the class `Throwable` or one of its subclasses. Throwing an exception results in an immediate nonlocal transfer of control from the point where the exception was thrown.

Most exceptions occur synchronously as a result of an action by the thread in which they occur. An asynchronous exception, by contrast, can potentially occur at any point in the execution of a program. The Java Virtual Machine throws an exception for one of three reasons:

- An *athrow* instruction (§*athrow*) was executed.
- An abnormal execution condition was synchronously detected by the Java Virtual Machine. These exceptions are not thrown at an arbitrary point in the program, but only synchronously after execution of an instruction that either:
  - Specifies the exception as a possible result, such as:
    - When the instruction embodies an operation that violates the semantics of the Java programming language, for example indexing outside the bounds of an array.
    - When an error occurs in loading or linking part of the program.
  - Causes some limit on a resource to be exceeded, for example when too much memory is used.
- An asynchronous exception occurred because an internal error occurred in the Java Virtual Machine implementation (§6.3).

A Java Virtual Machine implementation may permit a small but bounded amount of execution to occur before an asynchronous exception is thrown. This delay is permitted to allow optimized code to detect and throw these exceptions at points where it is practical to handle them while obeying the semantics of the Java programming language.

A simple implementation might poll for asynchronous exceptions at the point of each control transfer instruction. Since a program has a finite size, this provides a bound on the total delay in detecting an asynchronous exception. Since no asynchronous exception will occur between control transfers, the code generator has some flexibility to reorder computation between control transfers for greater performance. The paper Polling Efficiently on Stock Hardware by Marc Feeley, Proc. 1993 Conference on Functional Programming and Computer Architecture, Copenhagen, Denmark, pp. 179–187, is recommended as further reading.

Exceptions thrown by the Java Virtual Machine are precise: when the transfer of control takes place, all effects of the instructions executed before the point from which the exception is thrown must appear to have taken place. No instructions that occur after the point from which the exception is thrown may appear to have been evaluated. If optimized code has speculatively executed some of the instructions which follow the point at which the exception occurs, such code must be prepared to hide this speculative execution from the user-visible state of the program.

Each method in the Java Virtual Machine may be associated with zero or more *exception handlers*. An exception handler specifies the range of offsets into the Java Virtual Machine code implementing the method for which the exception handler is active, describes the type of exception that the exception handler is able to handle, and specifies the location of the code that is to handle that exception. An exception matches an exception handler if the offset of the instruction that caused the exception is in the range of offsets of the exception handler and the exception type is the same class as or a subclass of the class of exception that the exception handler handles. When an exception is thrown, the Java Virtual Machine searches for a matching exception handler in the current method. If a matching exception handler is found, the system branches to the exception handling code specified by the matched handler.

If no such exception handler is found in the current method, the current method invocation completes abruptly (§2.6.5). On abrupt completion, the operand stack and local variables of the current method invocation are discarded, and its frame is popped, reinstating the frame of the invoking method. The exception is then rethrown in the context of the invoker's frame and so on, continuing up the method invocation chain. If no suitable exception handler is found before the top of the method invocation chain is reached, the execution of the thread in which the exception was thrown is terminated. Before termination of the thread, the uncaught exception is handled according to the following rules:

- If the thread has an uncaught exception handler set, then that handler is executed.
- Otherwise, the method `uncaughtException` is invoked for the `ThreadGroup` that is the parent of the thread. If the `ThreadGroup` and its parent `ThreadGroups` do not override `uncaughtException`, then the default handler's `uncaughtException` method is invoked.

The order in which the exception handlers of a method are searched for a match is important. Within a `class` file, the exception handlers for each method are stored in a table (§4.7.3). At run time, when an exception is thrown, the Java Virtual Machine searches the exception handlers of the current method in the order that they appear in the corresponding exception handler table in the `class` file, starting from the beginning of that table.

Note that the Java Virtual Machine does not enforce nesting of or any ordering of the exception table entries of a method. The exception handling semantics of the Java programming language are implemented only through cooperation with the compiler (§3.12). When `class` files are generated by some other means, the defined search procedure ensures that all Java Virtual Machine implementations will behave consistently.

2.11. Instruction Set Summary

A Java Virtual Machine instruction consists of a one-byte *opcode* specifying the operation to be performed, followed by zero or more *operands* supplying arguments or data that are used by the operation. Many instructions have no operands and consist only of an opcode.

Ignoring exceptions, the inner loop of a Java Virtual Machine interpreter is effectively

```
do {
    atomically calculate pc and fetch opcode at pc;
    if (operands) fetch operands;
    execute the action for the opcode;
} while (there is more to do);
```

The number and size of the operands are determined by the opcode. If an operand is more than one byte in size, then it is stored in *big-endian* order - high-order byte first. For example, an unsigned 16-bit index into the local variables is stored as two unsigned bytes, *byte1* and *byte2*, such that its value is (*byte1* << 8) | *byte2*.

The bytecode instruction stream is only single-byte aligned. The two exceptions are the *lookupswitch* and *tableswitch* instructions (§*lookupswitch*, §*tableswitch*), which are padded to force internal alignment of some of their operands on 4-byte boundaries.

The decision to limit the Java Virtual Machine opcode to a byte and to forgo data alignment within compiled code reflects a conscious bias in favor of compactness, possibly at the cost of some performance in naive implementations. A one-byte opcode also limits the size of the instruction set. Not assuming data alignment means that immediate data larger than a byte must be constructed from bytes at run time on many machines.

2.11.1. Types and the Java Virtual Machine

Most of the instructions in the Java Virtual Machine instruction set encode type information about the operations they perform. For instance, the *iload* instruction (§*iload*) loads the contents of a local variable, which must be an `int`, onto the operand stack. The *fload* instruction (§*fload*) does the same with a `float` value. The two instructions may have identical implementations, but have distinct opcodes.

For the majority of typed instructions, the instruction type is represented explicitly in the opcode mnemonic by a letter: *i* for an `int` operation, *l* for `long`, *s* for `short`, *b* for `byte`, *c* for `char`, *f* for `float`, *d* for `double`, and *a* for `reference`. Some instructions for which the type is unambiguous do not have a type letter in their mnemonic. For instance, *arraylength* always operates on an object that is an array. Some instructions, such as *goto*, an unconditional control transfer, do not operate on typed operands.

Given the Java Virtual Machine's one-byte opcode size, encoding types into opcodes places pressure on the design of its instruction set. If each typed instruction supported all of the Java Virtual Machine's run-time data types, there would be more instructions than could be represented in a byte. Instead, the instruction set of the Java Virtual Machine provides a reduced level of type support for certain operations. In other words, the instruction set is intentionally not orthogonal. Separate instructions can be used to convert between unsupported and supported data types as necessary.

Table 2.11.1-A summarizes the type support in the instruction set of the Java Virtual Machine. A specific instruction, with type information, is built by replacing the *T* in the instruction template in the opcode column by the letter in the type column. If the type column for some instruction template and type is blank, then no instruction exists supporting that type of operation. For instance, there is a load instruction for type `int`, *iload*, but there is no load instruction for type `byte`.

Note that most instructions in Table 2.11.1-A do not have forms for the integral types `byte`, `char`, and `short`. None have forms for the `boolean` type. Whenever values of types `byte` and `short` are loaded onto the operand stack, they are implicitly converted by sign extension to values of type `int`. Similarly, whenever values of types `boolean` and `char` are loaded onto the operand stack, they are implicitly converted by zero extension to values of type `int`. Thus, most operations on values originally stored in variables of types `boolean`, `byte`, `char`, and `short` are correctly performed by instructions operating on values of computational type `int`.

Table 2.11.1-A. Type support in the Java Virtual Machine instruction set

opcode	byte	short	int	long	float	double	char	reference
Tipush	*bipush*	*sipush*						
Tconst			*iconst*	*lconst*	*fconst*	*dconst*		*aconst*
Tload			*iload*	*lload*	*fload*	*dload*		*aload*
Tstore			*istore*	*lstore*	*fstore*	*dstore*		*astore*
Tinc			*iinc*					
Taload	*baload*	*saload*	*iaload*	*laload*	*faload*	*daload*	*caload*	*aaload*
Tastore	*bastore*	*sastore*	*iastore*	*lastore*	*fastore*	*dastore*	*castore*	*aastore*
Tadd			*iadd*	*ladd*	*fadd*	*dadd*		
Tsub			*isub*	*lsub*	*fsub*	*dsub*		
Tmul			*imul*	*lmul*	*fmul*	*dmul*		
Tdiv			*idiv*	*ldiv*	*fdiv*	*ddiv*		
Trem			*irem*	*lrem*	*frem*	*drem*		
Tneg			*ineg*	*lneg*	*fneg*	*dneg*		
Tshl			*ishl*	*lshl*				
Tshr			*ishr*	*lshr*				
Tushr			*iushr*	*lushr*				
Tand			*iand*	*land*				
Tor			*ior*	*lor*				
Txor			*ixor*	*lxor*				
i2T	*i2b*	*i2s*		*i2l*	*i2f*	*i2d*	*i2c*	
l2T			*l2i*		*l2f*	*l2d*		
f2T			*f2i*	*f2l*		*f2d*		
d2T			*d2i*	*d2l*	*d2f*			
Tcmp				*lcmp*				
Tcmpl					*fcmpl*	*dcmpl*		
Tcmpg					*fcmpg*	*dcmpg*		
if_TcmpOP			*if_icmpOP*					*if_acmpOP*
Treturn			*ireturn*	*lreturn*	*freturn*	*dreturn*		*areturn*

The mapping between Java Virtual Machine storage types and Java Virtual Machine computational types is summarized by Table 2.11.1-B.

Certain Java Virtual Machine instructions such as *pop* and *swap* operate on the operand stack without regard to type; however, such instructions are constrained to use only on values of certain categories of computational types, also given in Table 2.11.1-B.

Table 2.11.1-B. Storage and Computational types in the Java Virtual Machine

Storage type	Computational type	Category
`boolean`	`int`	1
`byte`	`int`	1

Storage type	Computational type	Category
`char`	`int`	1
`short`	`int`	1
`int`	`int`	1
`float`	`float`	1
`reference`	`reference`	1
`returnAddress`	`returnAddress`	1
`long`	`long`	2
`double`	`double`	2

2.11.2. Load and Store Instructions

The load and store instructions transfer values between the local variables (§2.6.1) and the operand stack (§2.6.2) of a Java Virtual Machine frame (§2.6):

- Load a local variable onto the operand stack: *iload*, *iload_<n>*, *lload*, *lload_<n>*, *fload*, *fload_<n>*, *dload*, *dload_<n>*, *aload*, *aload_<n>*.
- Store a value from the operand stack into a local variable: *istore*, *istore_<n>*, *lstore*, *lstore_<n>*, *fstore*, *fstore_<n>*, *dstore*, *dstore_<n>*, *astore*, *astore_<n>*.
- Load a constant on to the operand stack: *bipush*, *sipush*, *ldc*, *ldc_w*, *ldc2_w*, *aconst_null*, *iconst_m1*, *iconst_<i>*, *lconst_<l>*, *fconst_<f>*, *dconst_<d>*.
- Gain access to more local variables using a wider index, or to a larger immediate operand: *wide*.

Instructions that access fields of objects and elements of arrays (§2.11.5) also transfer data to and from the operand stack.

Instruction mnemonics shown above with trailing letters between angle brackets (for instance, *iload_<n>*) denote families of instructions (with members *iload_0*, *iload_1*, *iload_2*, and *iload_3* in the case of *iload_<n>*). Such families of instructions are specializations of an additional generic instruction (*iload*) that takes one operand. For the specialized instructions, the operand is implicit and does not need to be stored or fetched. The semantics are otherwise the same (*iload_0* means the same thing as *iload* with the operand *0*). The letter between the angle brackets specifies the type of the implicit operand for that family of instructions: for *<n>*, a nonnegative integer; for *<i>*, an `int`; for *<l>*, a `long`; for *<f>*, a `float`; and for *<d>*, a `double`.

This notation for instruction families is used throughout this specification.

2.11.3. Arithmetic Instructions

The arithmetic instructions compute a result that is typically a function of two values on the operand stack, pushing the result back on the operand stack. There are two main kinds of arithmetic instructions: those operating on integer values and those operating on floating-point values. Within each of these kinds, the arithmetic instructions are specialized to Java Virtual Machine numeric types. There is no direct support for integer arithmetic on values of the `byte`, `short`, and `char` types (§2.11.1), or for values of the `boolean` type; those operations are handled by instructions operating on type `int`. Integer and floating-point instructions also differ in their behavior on overflow and divide-by-zero. The arithmetic instructions are as follows:

- Add: *iadd*, *ladd*, *fadd*, *dadd*.
- Subtract: *isub*, *lsub*, *fsub*, *dsub*.
- Multiply: *imul*, *lmul*, *fmul*, *dmul*.
- Divide: *idiv*, *ldiv*, *fdiv*, *ddiv*.
- Remainder: *irem*, *lrem*, *frem*, *drem*.
- Negate: *ineg*, *lneg*, *fneg*, *dneg*.
- Shift: *ishl*, *ishr*, *iushr*, *lshl*, *lshr*, *lushr*.
- Bitwise OR: *ior*, *lor*.
- Bitwise AND: *iand*, *land*.
- Bitwise exclusive OR: *ixor*, *lxor*.
- Local variable increment: *iinc*.
- Comparison: *dcmpg*, *dcmpl*, *fcmpg*, *fcmpl*, *lcmp*.

The semantics of the Java programming language operators on integer and floating-point values (JLS §4.2.2, JLS §4.2.4) are directly supported by the semantics of the Java Virtual Machine instruction set.

The Java Virtual Machine does not indicate overflow during operations on integer data types. The only integer operations that can throw an exception are the integer divide instructions (*idiv* and *ldiv*) and the integer remainder instructions (*irem* and *lrem*), which throw an `ArithmeticException` if the divisor is zero.

The Java Virtual Machine does not indicate overflow or underflow during operations on floating-point data types. That is, floating-point instructions never cause the Java Virtual Machine to throw a run-time exception (not to be confused with an IEEE 754 floating-point exception). An operation that overflows produces a signed infinity; an operation that underflows produces a subnormal value or a signed zero; an operation that has no unique mathematically defined result produces NaN. All numeric operations with NaN as an operand produce NaN as a result.

Comparisons on values of type `long` (*lcmp*) perform a signed comparison.

Comparisons on values of floating-point types (*dcmpg*, *dcmpl*, *fcmpg*, *fcmpl*) are performed using IEEE 754 nonsignaling comparisons.

2.11.4. Type Conversion Instructions

The type conversion instructions allow conversion between Java Virtual Machine numeric types. These may be used to implement explicit conversions in user code or to mitigate the lack of orthogonality in the instruction set of the Java Virtual Machine.

The Java Virtual Machine directly supports the following widening numeric conversions:

- `int` to `long`, `float`, or `double`
- `long` to `float` or `double`
- `float` to `double`

The widening numeric conversion instructions are *i2l*, *i2f*, *i2d*, *l2f*, *l2d*, and *f2d*. The mnemonics for these opcodes are straightforward given the naming conventions for typed instructions and the punning use of 2 to mean "to." For instance, the *i2d* instruction converts an `int` value to a `double`.

Most widening numeric conversions do not lose information about the overall magnitude of a numeric value. Indeed, conversions widening from `int` to `long` and `int` to `double` do not lose any information at all; the numeric value is preserved exactly. Conversions widening from `float` to `double` also preserve the numeric value exactly.

Conversions from `int` to `float`, or from `long` to `float`, or from `long` to `double`, may lose *precision*, that is, may lose some of the least significant bits of the value; the resulting floating-point value is a correctly rounded version of the integer value, using the round to nearest rounding policy (§2.8).

Despite the fact that loss of precision may occur, widening numeric conversions never cause the Java Virtual Machine to throw a run-time exception (not to be confused with an IEEE 754 floating-point exception).

A widening numeric conversion of an `int` to a `long` simply sign-extends the two's-complement representation of the `int` value to fill the wider format. A widening numeric conversion of a `char` to an integral type zero-extends the representation of the `char` value to fill the wider format.

Note that widening numeric conversions do not exist from integral types `byte`, `char`, and `short` to type `int`. As noted in §2.11.1, values of type `byte`, `char`, and `short` are internally widened to type `int`, making these conversions implicit.

The Java Virtual Machine also directly supports the following narrowing numeric conversions:

- `int` to `byte`, `short`, or `char`
- `long` to `int`
- `float` to `int` or `long`
- `double` to `int`, `long`, or `float`

The narrowing numeric conversion instructions are *i2b*, *i2c*, *i2s*, *l2i*, *f2i*, *f2l*, *d2i*, *d2l*, and *d2f*. A narrowing numeric conversion can result in a value of different sign, a different order of magnitude, or both; it may thereby lose precision.

A narrowing numeric conversion of an `int` or `long` to an integral type T simply discards all but the *n* lowest-order bits, where *n* is the number of bits used to represent type T. This may cause the resulting value not to have the same sign as the input value.

In a narrowing numeric conversion of a floating-point value to an integral type T, where T is either `int` or `long`, the floating-point value is converted as follows:

- If the floating-point value is NaN, the result of the conversion is an `int` or `long` `0`.
- Otherwise, if the floating-point value is not an infinity, the floating-point value is rounded to an integer value V using the round toward zero rounding policy (§2.8). There are two cases:
  - If T is `long` and this integer value can be represented as a `long`, then the result is the `long` value V.
  - If T is of type `int` and this integer value can be represented as an `int`, then the result is the `int` value V.
- Otherwise:
  - Either the value must be too small (a negative value of large magnitude or negative infinity), and the result is the smallest representable value of type `int` or `long`.
  - Or the value must be too large (a positive value of large magnitude or positive infinity), and the result is the largest representable value of type `int` or `long`.

A narrowing numeric conversion from `double` to `float` behaves in accordance with IEEE 754. The result is correctly rounded using the round to nearest rounding policy (§2.8). A value too small to be represented as a `float` is converted to a positive or negative zero of type `float`; a value too large to be represented as a `float` is converted to a positive or negative infinity. A `double` NaN is always converted to a `float` NaN.

Despite the fact that overflow, underflow, or loss of precision may occur, narrowing conversions among numeric types never cause the Java Virtual Machine to throw a run-time exception (not to be confused with an IEEE 754 floating-point exception).

2.11.5. Object Creation and Manipulation

Although both class instances and arrays are objects, the Java Virtual Machine creates and manipulates class instances and arrays using distinct sets of instructions:

- Create a new class instance: *new*.
- Create a new array: *newarray*, *anewarray*, *multianewarray*.
- Access fields of classes (`static` fields, known as class variables) and fields of class instances (non-`static` fields, known as instance variables): *getstatic*, *putstatic*, *getfield*, *putfield*.
- Load an array component onto the operand stack: *baload*, *caload*, *saload*, *iaload*, *laload*, *faload*, *daload*, *aaload*.
- Store a value from the operand stack as an array component: *bastore*, *castore*, *sastore*, *iastore*, *lastore*, *fastore*, *dastore*, *aastore*.
- Get the length of array: *arraylength*.
- Check properties of class instances or arrays: *instanceof*, *checkcast*.

2.11.6. Operand Stack Management Instructions

A number of instructions are provided for the direct manipulation of the operand stack: *pop*, *pop2*, *dup*, *dup2*, *dup_x1*, *dup2_x1*, *dup_x2*, *dup2_x2*, *swap*.

2.11.7. Control Transfer Instructions

The control transfer instructions conditionally or unconditionally cause the Java Virtual Machine to continue execution with an instruction other than the one following the control transfer instruction. They are:

- Conditional branch: *ifeq*, *ifne*, *iflt*, *ifle*, *ifgt*, *ifge*, *ifnull*, *ifnonnull*, *if_icmpeq*, *if_icmpne*, *if_icmplt*, *if_icmple*, *if_icmpgt* *if_icmpge*, *if_acmpeq*, *if_acmpne*.
- Compound conditional branch: *tableswitch*, *lookupswitch*.
- Unconditional branch: *goto*, *goto_w*, *jsr*, *jsr_w*, *ret*.

The Java Virtual Machine has distinct sets of instructions that conditionally branch on comparison with data of `int` and `reference` types. It also has distinct conditional branch instructions that test for the null reference and thus it is not required to specify a concrete value for `null` (§2.4).

Conditional branches on comparisons between data of types `boolean`, `byte`, `char`, and `short` are performed using `int` comparison instructions (§2.11.1). A conditional branch on a comparison between data of types `long`, `float`, or `double` is initiated using an instruction that compares the data and produces an `int` result of the comparison (§2.11.3). A subsequent `int` comparison instruction tests this result and effects the conditional branch. Because of its emphasis on `int` comparisons, the Java Virtual Machine provides a rich complement of conditional branch instructions for type `int`.

All `int` conditional control transfer instructions perform signed comparisons.

2.11.8. Method Invocation and Return Instructions

The following five instructions invoke methods:

- *invokevirtual* invokes an instance method of an object, dispatching on the (virtual) type of the object.
- *invokeinterface* invokes an interface method, searching the methods implemented by the particular run-time object to find the appropriate method.
- *invokespecial* invokes an instance method requiring special handling, either an instance initialization method (§2.9.1) or a method of the current class or its supertypes.
- *invokestatic* invokes a class (`static`) method in a named class.
- *invokedynamic* invokes the method which is the target of the call site object bound to the *invokedynamic* instruction. The call site object was bound to a specific lexical occurrence of the *invokedynamic* instruction by the Java Virtual Machine as a result of running a bootstrap method before the first execution of the instruction. Therefore, each occurrence of an *invokedynamic* instruction has a unique linkage state, unlike the other instructions which invoke methods.

The method return instructions, which are distinguished by return type, are *ireturn* (used for return types `boolean`, `byte`, `char`, `short`, and `int`), *lreturn*, *freturn*, *dreturn*, and *areturn*. In addition, the *return* instruction is used to return from methods declared to be void, instance initialization methods, and class or interface initialization methods.

2.11.9. Throwing Exceptions

An exception is thrown programmatically using the *athrow* instruction. Exceptions can also be thrown by various Java Virtual Machine instructions if they detect an abnormal condition.

2.11.10. Synchronization

The Java Virtual Machine supports synchronization of both methods and sequences of instructions within a method by a single synchronization construct: the *monitor*.

Method-level synchronization is performed implicitly, as part of method invocation and return (§2.11.8). A `synchronized` method is distinguished in the run-time constant pool's `method_info` structure (§4.6) by the `ACC_SYNCHRONIZED` flag, which is checked by the method invocation instructions. When invoking a method for which `ACC_SYNCHRONIZED` is set, the executing thread enters a monitor, invokes the method itself, and exits the monitor whether the method invocation completes normally or abruptly. During the time the executing thread owns the monitor, no other thread may enter it. If an exception is thrown during invocation of the `synchronized` method and the `synchronized` method does not handle the exception, the monitor for the method is automatically exited before the exception is rethrown out of the `synchronized` method.

Synchronization of sequences of instructions is typically used to encode the `synchronized` block of the Java programming language. The Java Virtual Machine supplies the *monitorenter* and *monitorexit* instructions to support such language constructs. Proper implementation of `synchronized` blocks requires cooperation from a compiler targeting the Java Virtual Machine (§3.14).

Structured locking is the situation when, during a method invocation, every exit on a given monitor matches a preceding entry on that monitor. Since there is no assurance that all code submitted to the Java Virtual Machine will perform structured locking, implementations of the Java Virtual Machine are permitted but not required to enforce both of the following two rules guaranteeing structured locking. Let *T* be a thread and *M* be a monitor. Then:

1. The number of monitor entries performed by *T* on *M* during a method invocation must equal the number of monitor exits performed by *T* on *M* during the method invocation whether the method invocation completes normally or abruptly.
2. At no point during a method invocation may the number of monitor exits performed by *T* on *M* since the method invocation exceed the number of monitor entries performed by *T* on *M* since the method invocation.

Note that the monitor entry and exit automatically performed by the Java Virtual Machine when invoking a `synchronized` method are considered to occur during the calling method's invocation.

2.12. Class Libraries

The Java Virtual Machine must provide sufficient support for the implementation of the class libraries of the Java SE Platform. Some of the classes in these libraries cannot be implemented without the cooperation of the Java Virtual Machine.

Classes that might require special support from the Java Virtual Machine include those that support:

- Reflection, such as the classes in the package `java.lang.reflect` and the class `Class`.
- Loading and creation of a class or interface. The most obvious example is the class `ClassLoader`.
- Linking and initialization of a class or interface. The example classes cited above fall into this category as well.
- The module system, such as the class `ModuleLayer`.
- Multithreading, such as the class `Thread`.
- Weak references, such as the classes in the package `java.lang.ref`.

The list above is meant to be illustrative rather than comprehensive. An exhaustive list of these classes or of the functionality they provide is beyond the scope of this specification. See the specifications of the Java SE Platform class libraries for details.

2.13. Public Design, Private Implementation

Thus far this specification has sketched the public view of the Java Virtual Machine: the `class` file format and the instruction set. These components are vital to the hardware-, operating system-, and implementation-independence of the Java Virtual Machine. The implementor may prefer to think of them as a means to securely communicate fragments of programs between hosts each implementing the Java SE Platform, rather than as a blueprint to be followed exactly.

It is important to understand where the line between the public design and the private implementation lies. A Java Virtual Machine implementation must be able to read `class` files and must exactly implement the semantics of the Java Virtual Machine code therein. One way of doing this is to take this document as a specification and to implement that specification literally. But it is also perfectly feasible and desirable for the implementor to modify or optimize the implementation within the constraints of this specification. So long as the `class` file format can be read and the semantics of its code are maintained, the implementor may implement these semantics in any way. What is "under the hood" is the implementor's business, as long as the correct external interface is carefully maintained.

There are some exceptions: debuggers, profilers, and just-in-time code generators can each require access to elements of the Java Virtual Machine that are normally considered to be "under the hood." Where appropriate, Oracle works with other Java Virtual Machine implementors and with tool vendors to develop common interfaces to the Java Virtual Machine for use by such tools, and to promote those interfaces across the industry.

The implementor can use this flexibility to tailor Java Virtual Machine implementations for high performance, low memory use, or portability. What makes sense in a given implementation depends on the goals of that implementation. The range of implementation options includes the following:

- Translating Java Virtual Machine code at load-time or during execution into the instruction set of another virtual machine.
- Translating Java Virtual Machine code at load-time or during execution into the native instruction set of the host CPU (sometimes referred to as *just-in-time*, or *JIT*, code generation).

The existence of a precisely defined virtual machine and object file format need not significantly restrict the creativity of the implementor. The Java Virtual Machine is designed to support many different implementations, providing new and interesting solutions while retaining compatibility between implementations.